

整合練習 7：DBCO 與 RFC Destination 設定

Lecture

if06 講了「RFC-enabled FM 讓別人呼叫我方系統」；這題講反過來——我方系統要主動呼叫外部系統，需要先設定「要連去哪裡」，這正是 **SM59 (RFC Destination)** 跟 **DBCO (Secondary Database Connection)** 的角色。這兩個交易碼很常被搞混，這題先把兩者的定位釐清，再看實際查證到的系統資料。

SM59——RFC Destination，系統對系統的呼叫目標：定義「呼叫一個 RFC 目標時，要用什麼協定、連去哪台主機、用哪個帳號登入」。底層資料表是 **RFCDES**，這次直接查證讀到這套系統實際配置的內容 (**RFCTYPE = '3'** 篩選出「ABAP Connection」型別，即連到另一套 SAP ABAP 系統的目標)：

RFCDEST	RFCTYPE
ABACLNT800	3
BGRFC_SUPERVISOR	3
BG_RFC_MM	3
DYNAMIC_DEST_CALLBACK_WHITELIST	3
FINBTR@S4HCLNT000	3
FINBTR@S4HCLNT100	3
IWNGW_BGRFC	3
LOCAL_RFC	3
MDGTR@S4HCLNT100	3
N74CLNT100	3
N74CLNT100_trust	3
NW_RFC	3
S4FIN_RFC	3
S4H	3
S4HCLNT100	3
... (總共 48 筆)	

這套系統確實配置了一批 RFC Destination (多半是 S/4HANA 標準場景自帶的內部/自我參照連線，如 **LOCAL_RFC**、各種 ***CLNT*** 命名的連到特定 Client 的連線)。 **RFCOPTIONS** 欄位存著連線細節，用逼近 **key=value** 的緊湊編碼格式 (例如 **S=00,M=800,U=IDADMIN,L=E,...**，**S** = System Number、**M** = Client、**U** = User、**L** = Logon Language 這類)，這也是為什麼 **SAP** 從來不建議直接改這張表——SM59 畫面存在的目的就是要把這串編碼包成人看得懂的表單，直接動 **RFCDES** 表本身既不支援 (沒有 Update 邏輯、也沒有密碼加密處理) 也極度不建議。

除了 Type 3 (ABAP Connection) 之外，SM59 常見的其他 Connection Type 還有：**T** (TCP/IP，用來連外部程式，例如透過 RFC SDK/JCo 寫的非 ABAP 程式)、**H** (HTTP 連線)、**L** (Logical Destination，邏輯目的地，實際連線設定另外查表)——這題只查證了 Type 3 的部分，其餘型別留待有需要時再實測補充。

DBCO——Secondary Database Connection，資料庫層的連線：if06 提到的 Native SQL、if08 要教的 ADBC，預設都是連「目前登入的這個資料庫」 (Standard Connection)；如果要連另一個資料庫 (可能是另一套 SAP 系統的資料庫、也可能是完全不同廠牌的異質資料庫)，要先在 DBCO 交易碼設定一個 Secondary Database Connection，之後 Native SQL 用 **CONNECT** / ADBC 用 **cl_sql_connection=>get_connection(**

dbcon) 才能指名連過去。底層表是 **DBCON**——這次實測發現這張表比 **RFCDES** 保護得更嚴格，連讀取都被系統擋下：

You cannot display DBCON with the standard tools

(用跟查 **RFCDES** 完全一樣的唯一讀 SQL 查詢工具去查 **DBCON**，直接被系統擋掉，這是系統主動的保護機制，不是工具的限制——**DBCON** 可能存有連線密碼等敏感資訊，SAP 刻意不讓一般的資料預覽 / SQL 查詢工具讀取這張表，這比 **RFCDES** 只是「不建議直接改」更進一步，是「連讀都不給讀」。)

兩者關係一句話釐清：**RFC Destination** 是「系統對系統」的呼叫目標（呼叫一支遠端 **FM**）；**DBCO** 是「資料庫對資料庫」的連線（直接下 **SQL** 到另一個資料庫，不透過對方系統的 **ABAP** 應用邏輯）——這是很多人搞混的地方：以為「連到另一套 SAP 系統」只有一種做法，實際上要看目的是「呼叫對方的 **FM** / **BAPI**」（用 **RFC Destination**）還是「繞過對方應用邏輯、直接讀寫對方的資料庫」（用 **DBCO**，通常只在你自己有權限、且清楚在做什麼的情境才會這樣做，因為繞過應用邏輯的風險跟 if04 教過的「Open SQL 直寫繞過業務規則」是同一類問題）。

⚠ 已知限制：SM59 / DBCO 都是傳統 Dynpro 交易，這次查證 ADT **discovery** 全文，完全沒有找到跟 **destination** / **dbcon** / **rfcdest** 相關的 collection——確認屬於本專案已知的 GUI-only 類別（跟 Search Help、T-code、BOR、LSMW 同一種限制）。這題能做到的是讀取底層表確認現況（**RFCDES** 讀得到、**DBCON** 讀不到），沒辦法透過任何 API 建立或修改 RFC Destination / Database Connection，這兩項設定只能靠 SAP GUI 手動操作。

學習目標

- 能講出 RFC Destination (SM59) 跟 DBCO 的本質差異：系統對系統呼叫 vs 資料庫對資料庫連線
- 知道 SM59 的常見 Connection Type (至少 3 = ABAP Connection、T = TCP/IP、H = HTTP)
- 理解為什麼不該直接改 **RFCDES** / **DBCON** 這兩張底層表，即使技術上（部分）讀得到
- 認清這題屬於 GUI-only 限制，Claude 端只能做到「讀取現況」，設定本身要靠使用者在 SAP GUI 操作

事前準備

不需要新建任何物件，這題是唯讀查證 + 觀念釐清。

題目需求

1. 用本題查到的 **RFCDES** 資料，指出這套系統裡至少 3 個 Type 3 (ABAP Connection) 的目的地名稱。
2. 解釋為什麼 **DBCON** 表連讀取都被系統擋下，但 **RFCDES** 可以讀（雖然依然不建議直接改）——兩者存放的資訊敏感程度有什麼差異？
3. 情境判斷：如果需求是「每天定時去另一套 SAP 系統呼叫一支 BAPI 同步資料」，該設定 RFC Destination 還是 DBCO？如果需求是「直接讀取另一個異質資料庫（例如一個舊系統的 Oracle DB）的一張表」呢？

參考答案 (情境判斷)

- 呼叫另一套 SAP 系統的 **BAPI**：用 **RFC Destination** (SM59，Type 3)，程式端用 **CALL FUNCTION** 'xxx' DESTINATION 'yyy' ... 指名目的地呼叫。
- 直接讀取異質資料庫的一張表：用 **DBCO** 設定 Secondary Database Connection，程式端搭配 Native SQL 的 **CONNECT** / if08 要教的 **ADBC cl_sql_connection=>get_connection(dbcon)** 指名連過去，

繞過對方系統的應用邏輯直接下 SQL。

團隊實務備註

- **RFCDES / DBCON** 兩張表的查證都是用 `/sap/bc/adt/datapreview/freestyle` 端點 (if05 開發過程發現的方法，見 `.claude/rules/sap-adt-mcp.md` 第 18 節)，對 **RFCDES** 成功、對 **DBCON** 收到系統主動擋下的例外訊息——這本身就是很好的第一手教材，不用另外找文件佐證「**DBCON** 比較敏感」這件事。
- 這次已確認 **ADT discovery** 全文沒有 **SM59 / DBCO** 相關的 collection，補齊了 **README** 草案階段標註「待實測確認」的項目——結論是兩者都屬於 **GUI-only**，`.claude/rules/sap-adt-mcp.md` 目前沒有為此開新章節記錄 (跟已經記錄的 **Search Help / T-code / BOR / LSMW** 屬於同一類已知模式，不需要每個都重複記一次，這裡直接沿用既有結論)。
- **RFCOPTIONS** 欄位的編碼細節 (**S=/M=/U=/L=** 等單字母代碼) 這次只做了粗略解讀，沒有逐一查證每個代碼的正式定義——這題的重點是「不要直接碰這張表」，不是「精通這串編碼格式」，出題時不需要深究到能手動解讀完整字串的程度。
- **⚠ 修正 (if08 開發時發現)**：上面第 2 點「**DBCON** 連讀取都被系統擋下」這句話容易被誤解成「**DBCON** 是張連 **Open SQL** 都讀不到的表」——這不對。if08 讀 `CL_SQL_CONNECTION=>get_connection` 的原始碼時，看到它內部驗證連線名稱是否存在，用的就是最普通的 `SELECT SINGLE dbms FROM dbcon WHERE con_name = con_name`，代表 **DBCON** 完全是張可以被 **ABAP** 程式正常 **SELECT** 的表。真正被擋下的只有 `/sap/bc/adt/datapreview/freestyle` 這個資料預覽便利工具，它自己刻意加了保護、拒絕顯示 **DBCON** 內容，這是工具層級的保護，不是資料庫權限或 **Open SQL** 層級的限制。上面問題 2 的正確理解應該是「這張表存的資訊敏感到連『唯讀資料預覽』這種通常什麼表都給看的工具都特別排除它」，而不是「這張表在程式碼層級也讀不到」。詳見 if08 Lecture 與團隊實務備註。

思考題

1. 如果一個 **RFC Destination** 的登入方式是「儲存的使用者帳密」(**Stored Logon**)，而不是「目前登入使用者的 **Trusted** 連線」，這對安全性有什麼取捨？(提示：儲存帳密代表任何能執行這支呼叫程式的人都繼承了那組帳密的權限，不需要額外驗證；**Trusted** 連線則是把「目前使用者是誰」傳遞過去，由對方系統決定授權，安全性設計哲學不同，這也是 **SM59** 設定時要考慮的重點之一，這題沒有深入實作，只需要能講出取捨)
2. **DBCON** 讀不到，那如果真的需要在程式裡確認「這個 **Secondary Connection** 到底存不存在」，該怎麼做？(提示：不透過查表，改用程式層級的方式測試——例如實際嘗試用 if08 的 `ADBC cl_sql_connection=>get_connection( dbcon )` 去連連看，用 `TRY...CATCH cx_sql_exception` 判斷連不連得上，這是「用行為驗證存在，而不是直接查設定資料」的思路，跟 if02 學到的「有時候要用旁敲側擊的方式確認一件事」是同一種手法的另一種應用)
3. 這題查到的 48 個 **Type 3 destination** 裡，有些命名帶著 `@S4HCLNT000 / @S4HCLNT100` 這種格式，你猜這個命名慣例在表達什麼？(提示：`@` 後面接的很像是「系統代號 + Client 號碼」，這種命名慣例讓管理員一眼就知道這個 **destination** 連到哪個系統的哪個 **Client**，是命名規範帶來的可讀性，不是系統強制的語法)